Advanced NLP Models Demo

https://huggingface.co/docs

In [ ]:
```python
import warnings

import numpy as np
import torch
import torch.nn as nn
from peft import LoraConfig, TaskType, get_peft_model
from sklearn.metrics import accuracy_score
from torch.utils.data import Dataset
from transformers import (
    AutoModelForSequenceClassification,
    AutoTokenizer,
    CLIPModel,
    CLIPProcessor,
    Trainer,
    TrainingArguments,
    pipeline,
)

warnings.filterwarnings("ignore")
```

Why does AutoTokenizer exist?

Different model families use completely different tokenization algorithms and vocab

Tokenizers — Two Worked Examples

Transformers don't read text — they read **integer IDs**. A *tokenizer* converts strings ↔ IDs. Different model families use different tokenizers, so the same sentence becomes different numbers depending on the model.

Input: `"I love transformers! 🤗 AGI is coming in 2026."`

---

1. BERT — WordPiece (uncased)

```python
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("bert-base-uncased")
tok.tokenize("I love transformers! 🤗 AGI is coming in 2026.")
```

Output:

```
['i', 'love', 'transformers', '!', '[UNK]', 'agi', 'is', 'coming', 'in', '2026', '.']
```

- Lowercases everything: `"I"` → `"i"`, `"AGI"` → `"agi"` (case info **lost**).
- Emoji 🤗 → `[UNK]` (**information lost** — vocab is closed).
- Auto-adds `[CLS]` (101) at the start and `[SEP]` (102) at the end.
- Continuation marker for split words: `##` (e.g. `transformer + ##s`).

---

2. GPT-2 — byte-level BPE

```python
tok = AutoTokenizer.from_pretrained("gpt2")
tok.tokenize("I love transformers! 🤗 AGI is coming in 2026.")
```

Output:

```
['I', 'Ġlove', 'Ġtransformers', '!', 'ĠðŁ', '¤', 'Ĺ', 'ĠAG', 'I', 'Ġis', 'Ġcoming', 'Ġin', 'Ġ2026',
'.']
```

- Case preserved.
- `Ġ` = "this token starts with a space" — whitespace is never lost.
- Emoji 🤗 → 3 byte-tokens (its UTF-8 bytes). **Never produces** `[UNK]` — fully reversible.
- `"AGI"` → `ĠAG` + `I` (no merge for this combo).
- No `[CLS]` / `[SEP]`.

Side-by-side

Property	BERT (WordPiece)	GPT-2 (byte-BPE)
Case preserved	❌ (uncased)	✅
Emoji 🤗	`[UNK]` (lost)	byte-tokens (recoverable)
`[CLS]` / `[SEP]` added	✅	❌
Continuation marker	`##xyz`	`Ġxyz` (= leading space)
Reversible?	❌	✅
Vocab size	30,522	50,257

Key takeaways

1. **Tokenizer must match the model** — IDs are a private contract with the embedding matrix.
2. **Same string → different token counts** for different tokenizers.
3. **BERT is lossy** (lowercases, `[UNK]`); **byte-level BPE is lossless.**
4. Tokens are **subwords** — a learned compromise based on training-data frequency.

Subword tokenizers learn merges from a training corpus. **Frequent substrings → 1 token. Rare ones → split into pieces.**

(a) Common vs rare English words

```
from transformers import AutoTokenizer
gpt = AutoTokenizer.from_pretrained("gpt2")

gpt.tokenize("hello")
# ['hello']                                                    ← 1 token

gpt.tokenize("antidisestablishmentarianism")
# ['ant', 'idis', 'establishment', 'arian', 'ism']             ← 5 tokens
```

(b) Non-English text costs more tokens

```
gpt.tokenize("こんにちは")
# ['ãģ', 'ĵ', 'ãĤ', 'ãģ', '«', 'ãģ', '¡', 'ãģ', 'ī']          ← 9 byte-tokens
```

GPT-2's corpus was mostly English, so Japanese falls back to raw UTF-8 bytes. That's why GPT-4 inference on Japanese costs **~2–3× the tokens** of equivalent English — purely an artifact of the training data, not the language.

TEXT CLASSIFICATION AND SENTIMENT ANALYSIS

https://huggingface.co/docs/transformers/en/main_classes/pipelines

In [ ]:
```
sentiment_pipeline = pipeline(
    "sentiment-analysis", model="cardiffnlp/twitter-roberta-base-sentiment-latest"
)

texts = [
    "I love this new transformer model!",
    "This is terrible, I hate it.",
    "The weather is okay today, nothing special.",
    "Machine learning is revolutionizing the world!",
]

for text in texts:
    result = sentiment_pipeline(text)
    print(f"Text: {text}")
    print(f"Sentiment: {result[0]['label']} (confidence: {result[0]['score']:.3f})\n")
```

CUSTOM TEXT CLASSIFICATION WITH TRANSFER LEARNING

In [6]:
```
class CustomTextClassificationDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=128):
        self.texts = texts
```

```
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        encoding = self.tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=self.max_length,
            return_tensors="pt",
        )

        return {
            "input_ids": encoding["input_ids"].flatten(),
            "attention_mask": encoding["attention_mask"].flatten(),
            "labels": torch.tensor(self.labels[idx], dtype=torch.long),
        }
```

In [ ]:
```
# Load model and tokenizer
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=2, output_attentions=False, output_hidden_states=False
)
```

In [ ]:
```
model
```

In [9]:
```
# Sample data (in practice, you'd load your own dataset)
sample_texts = [
    "This product is amazing and works perfectly!",
    "Terrible quality, broke after one day.",
    "Great value for money, highly recommend.",
    "Worst purchase ever, complete waste of money.",
]
sample_labels = [1, 0, 1, 0]  # 1: positive, 0: negative

# Create dataset
dataset = CustomTextClassificationDataset(sample_texts, sample_labels, tokenizer)
```

In [ ]:
```
dataset[0]
```

In [ ]:
```
# Print number of trainable parameters
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Number of trainable parameters: {trainable_params:,}")

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=20,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    logging_dir="./logs",
    logging_steps=10,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    eval_dataset=dataset,
)
```

In [ ]:
```
trainer.train()
```

In [ ]:
```
# Evaluate the trained model
print("\n=== Evaluating Trained Model ===")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Create evaluation dataset with 4 samples
eval_texts = [
    "The service was excellent and very professional",
    "I'm disappointed with the quality of this product",
    "This exceeded my expectations, great purchase!",
    "Not worth the money, poor performance",
    "Awful awful service, I'm never coming back",
```

```
]
eval_labels = [1, 0, 1, 0, 0]  # 1: positive, 0: negative

# Create evaluation dataset
eval_dataset = CustomTextClassificationDataset(eval_texts, eval_labels, tokenizer)

# Set model to evaluation mode
model.eval()

# Create evaluation dataloader
eval_dataloader = torch.utils.data.DataLoader(eval_dataset, batch_size=2, shuffle=False)

# Lists to store predictions and true labels
all_predictions = []
all_labels = []

# Evaluate without gradient computation
with torch.no_grad():
    for batch in eval_dataloader:
        # Move batch to device
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        # Get model predictions
        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        predictions = torch.argmax(outputs.logits, dim=-1)

        # Store predictions and labels
        all_predictions.extend(predictions.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Calculate accuracy
accuracy = accuracy_score(all_labels, all_predictions)
print(f"Model Accuracy: {accuracy:.4f}")

# Print detailed results
print("\nDetailed Results:")
for i, (text, true_label, pred) in enumerate(
    zip(sample_texts, all_labels, all_predictions)
):
    print(f"\nSample {i + 1}:")
    print(f"Text: {text}")
    print(f"True Label: {'Positive' if true_label == 1 else 'Negative'}")
    print(f"Predicted: {'Positive' if pred == 1 else 'Negative'}")
```

CUSTOM TRAINING LOOP

In [ ]:

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=2, output_attentions=False, output_hidden_states=False
)
model.to(device)

# Training parameters
num_epochs = 20
batch_size = 8
learning_rate = 2e-4
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

# Create DataLoader
train_dataloader = torch.utils.data.DataLoader(
    dataset, batch_size=batch_size, shuffle=True
)

# Training loop
model.train()
for epoch in range(num_epochs):
    total_loss = 0
    for batch in train_dataloader:
        # Move batch to device
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        # Forward pass
        outputs = model(
            input_ids=input_ids, attention_mask=attention_mask, labels=labels
        )
        loss = outputs.loss
```

```
        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_dataloader)
    print(f"Epoch {epoch + 1}/{num_epochs}, Average Loss: {avg_loss:.4f}")
```

LORA TRAINING LOOP

https://huggingface.co/docs/peft/en/index

In [ ]:
```
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=2, output_attentions=False, output_hidden_states=False
)
model.to(device)
# Configure LoRA
lora_config = LoraConfig(
    r=16,  # rank
    lora_alpha=32,
    target_modules=["q_lin", "v_lin"],  # target attention layers
    lora_dropout=0.05,
    bias="none",
    task_type="SEQ_CLS",
)

# Prepare model for LoRA fine-tuning
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, lora_config)

print(model)

# Configure training arguments
training_args = TrainingArguments(
    output_dir="./lora_results",
    num_train_epochs=20,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    logging_dir="./lora_logs",
    logging_steps=10,
    save_strategy="epoch",
    learning_rate=2e-4,
    fp16=True,  # Enable mixed precision training
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    eval_dataset=dataset,
)
```

In [ ]:
```
# Print number of trainable parameters
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Number of trainable parameters: {trainable_params:,}")
```

In [ ]:
```
trainer.train()
```

QUESTION ANSWERING

In [ ]:
```
qa_pipeline = pipeline(
    "question-answering", model="distilbert-base-cased-distilled-squad"
)

context = """
The transformer architecture was introduced in the paper "Attention Is All You Need" by Vaswani et al. in 2017.
It revolutionized natural language processing by using self-attention mechanisms instead of recurrent layers.
The model consists of an encoder and decoder, each made up of multiple layers with multi-head attention and
feed-forward networks. BERT, GPT, and T5 are all based on the transformer architecture.
"""

questions = [
```

```
    "When was the transformer architecture introduced?",
    "What did the transformer architecture replace?",
    "What are some models based on transformer architecture?",
]

for question in questions:
    answer = qa_pipeline(question=question, context=context)
    print(f"Q: {question}")
    print(f"A: {answer['answer']} (confidence: {answer['score']:.3f})\n")
```

CLIP

https://openai.com/index/clip/

In [ ]:

```
from PIL import Image

# Load CLIP model
clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
clip_processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# Load and process the cat image
image = Image.open("cat.jpg")
image_inputs = clip_processor(images=image, return_tensors="pt", padding=True)

# Text descriptions to compare with the image
text_descriptions = [
    "a photo of a cat",
    "a photo of a dog",
    "a photo of a car",
    "a photo of a bicycle",
    "a photo of a bird",
]

# Get image and text features
image_features = clip_model.get_image_features(**image_inputs)
text_inputs = clip_processor(text=text_descriptions, return_tensors="pt", padding=True)
text_features = clip_model.get_text_features(**text_inputs)

# Compute similarity between image and text descriptions
similarity_scores = torch.cosine_similarity(image_features, text_features, dim=1)

# Print results
print("\nSimilarity scores between cat image and text descriptions:")
for text, score in zip(text_descriptions, similarity_scores):
    print(f"{text}: {score.item():.4f}")
```